** FRAUD DETECTION FRAUD DETECTION MODEL **

In [190…

```python
# 1. IMPORT LIBRARIES
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

from imblearn.over_sampling import SMOTE
```

In [192…

```python
# 2. LOAD DATA
df = pd.read_csv('credit_card_fraud_dataset.csv')
df.head()
```

Out[192…

	TransactionID	TransactionDate	Amount	MerchantID	TransactionType	Location	IsFraud
0	1	2024-04-03 14:15:35.462794	4189.27	688	refund	San Antonio	0
1	2	2024-03-19 13:20:35.462824	2659.71	109	refund	Dallas	0
2	3	2024-01-08 10:08:35.462834	784.00	394	purchase	New York	0
3	4	2024-04-13 23:50:35.462850	3514.40	944	purchase	Philadelphia	0
4	5	2024-07-12 18:51:35.462858	369.07	475	purchase	Phoenix	0

In [194…

```python
# 3. BASIC INFO
print("Shape:", df.shape)
print("\nColumns:", df.columns)
```

```
Shape: (100000, 7)

Columns: Index(['TransactionID', 'TransactionDate', 'Amount', 'MerchantID',
       'TransactionType', 'Location', 'IsFraud'],
      dtype='object')
```

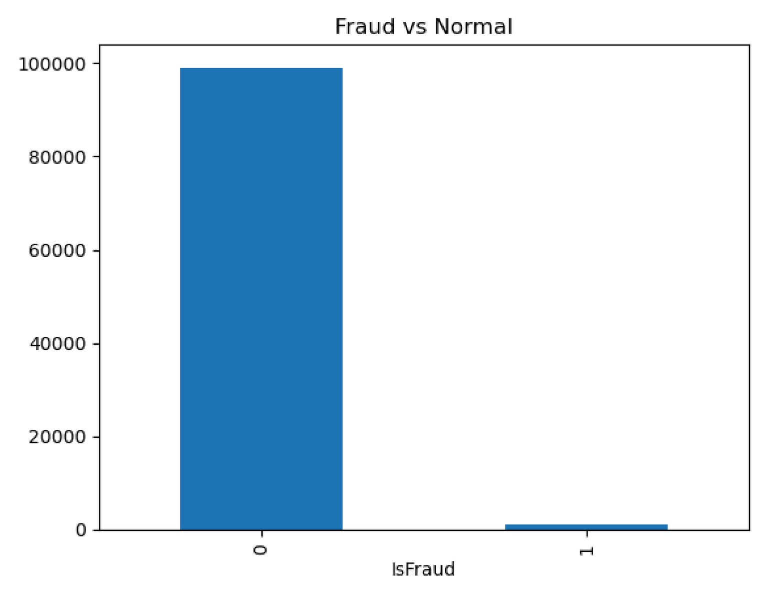
In [196…

```python
# 4. TARGET DISTRIBUTION
print("\nTarget Distribution:\n", df['IsFraud'].value_counts())

df['IsFraud'].value_counts().plot(kind='bar', title="Fraud vs Normal")
```

```
Target Distribution:
 IsFraud
0    99000
1     1000
Name: count, dtype: int64
```

Out[196… `<Axes: title={'center': 'Fraud vs Normal'}, xlabel='IsFraud'>`



In [198…

```python
# 5. DATE PROCESSING
df['TransactionDate'] = pd.to_datetime(df['TransactionDate'])

df['Year'] = df['TransactionDate'].dt.year
df['Month'] = df['TransactionDate'].dt.month
df['Day'] = df['TransactionDate'].dt.day

df.drop('TransactionDate', axis=1, inplace=True)
df.head()
```

Out[198…

	TransactionID	Amount	MerchantID	TransactionType	Location	IsFraud	Year	Month	Day
0	1	4189.27	688	refund	San Antonio	0	2024	4	3
1	2	2659.71	109	refund	Dallas	0	2024	3	19
2	3	784.00	394	purchase	New York	0	2024	1	8
3	4	3514.40	944	purchase	Philadelphia	0	2024	4	13
4	5	369.07	475	purchase	Phoenix	0	2024	7	12

In [200…

```python
# 6. ENCODING CATEGORICAL FEATURES
df = pd.get_dummies(df, columns=['TransactionType', 'Location'], drop_first=True)
df.head()
```

Out[200…

	TransactionID	Amount	MerchantID	IsFraud	Year	Month	Day	TransactionType_refund	Location_Dallas	Location_Houston	Location_Los Angeles	Location_New York	Location_Philadelphia	Location_Phoenix	Location_San Antonio	Location_San Diego	Locatio
0	1	4189.27	688	0	2024	4	3	True	False	False	False	False	False	False	True	False	
1	2	2659.71	109	0	2024	3	19	True	True	False	False	False	False	False	False	False	
2	3	784.00	394	0	2024	1	8	False	False	False	False	True	False	False	False	False	
3	4	3514.40	944	0	2024	4	13	False	False	False	False	False	True	False	False	False	
4	5	369.07	475	0	2024	7	12	False	False	False	False	False	False	True	False	False	

In [202…

```python
# 7. SPLIT FEATURES & TARGET
X = df.drop('IsFraud', axis=1)
y = df['IsFraud']
X,y
```

Out[202…

```
(       TransactionID   Amount  MerchantID  Year  Month  Day  \
 0                  1  4189.27         688  2024      4    3
 1                  2  2659.71         109  2024      3   19
 2                  3   784.00         394  2024      1    8
 3                  4  3514.40         944  2024      4   13
 4                  5   369.07         475  2024      7   12
 ...              ...      ...         ...   ...    ...  ...
 99995          99996  1057.29         289  2024      6    7
 99996          99997   297.25         745  2023     10   22
 99997          99998  3448.56         690  2024      5   31
 99998          99999  3750.79         644  2024     10   18
 99999         100000  1596.79         675  2024      3    5

        TransactionType_refund  Location_Dallas  Location_Houston  \
 0                        True            False             False
 1                        True             True             False
 2                       False            False             False
 3                       False            False             False
 4                       False            False             False
 ...                       ...              ...               ...
 99995                    True            False             False
 99996                    True            False             False
 99997                   False            False             False
 99998                   False            False             False
 99999                    True            False              True

        Location_Los Angeles  Location_New York  Location_Philadelphia  \
 0                     False              False                  False
 1                     False              False                  False
 2                     False               True                  False
 3                     False              False                   True
 4                     False              False                  False
 ...                     ...                ...                    ...
 99995                 False              False                  False
 99996                 False              False                  False
 99997                 False              False                  False
 99998                 False              False                   True
 99999                 False              False                  False

        Location_Phoenix  Location_San Antonio  Location_San Diego  \
 0                 False                  True               False
 1                 False                 False               False
 2                 False                 False               False
 3                 False                 False               False
 4                  True                 False               False
 ...                 ...                   ...                 ...
 99995             False                  True               False
 99996             False                  True               False
 99997             False                  True               False
 99998             False                 False               False
 99999             False                 False               False

        Location_San Jose
 0                  False
 1                  False
 2                  False
 3                  False
 4                  False
 ...                  ...
 99995              False
 99996              False
 99997              False
 99998              False
 99999              False

 [100000 rows x 16 columns],
 0        0
 1        0
 2        0
 3        0
 4        0
         ..
 99995    0
 99996    0
 99997    0
 99998    0
 99999    0
 Name: IsFraud, Length: 100000, dtype: int64)
```

In [204…

```python
# 8. TRAIN-TEST SPLIT
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

In [206…

```python
# 9. HANDLE IMBALANCE (SMOTE)
sm = SMOTE(random_state=42)
X_train_res, y_train_res = sm.fit_resample(X_train, y_train)

print("\nBefore SMOTE:\n", y_train.value_counts())
print("\nAfter SMOTE:\n", pd.Series(y_train_res).value_counts())
```

```
Before SMOTE:
 IsFraud
0    79200
1      800
Name: count, dtype: int64

After SMOTE:
 IsFraud
0    79200
1    79200
Name: count, dtype: int64
```

In [208…

```python
# 10. SCALING (IMPORTANT FOR LR)
scaler = StandardScaler()
X_train_res = scaler.fit_transform(X_train_res)
X_test_scaled = scaler.transform(X_test)
X_train_res,X_test_scaled
```

Out[208…

```
(array([[-1.43627357, -0.10194675,  1.42312161, ..., -0.41705317,
         -0.44417109, -0.38577852],
        [ 1.43912791,  1.50152388,  1.81250937, ..., -0.41705317,
         -0.44417109,  2.59216092],
        [ 0.39584218,  0.59481351, -1.57856654, ..., -0.41705317,
         -0.44417109, -0.38577852],
        ...,
        [ 0.8166901 ,  0.63403057,  0.27386065, ...,  2.39777582,
         -0.44417109,  2.59216092],
        [-1.33986101,  1.06178152, -0.36125724, ..., -0.41705317,
          2.25138468, -0.38577852],
        [ 1.27443323,  1.22139894, -0.16845359, ..., -0.41705317,
         -0.44417109,  2.59216092]]),
 array([[-0.38314912, -1.32066229,  0.43642059, ..., -0.41705317,
         -0.44417109, -0.38577852],
        [-1.67493189,  0.94853463,  1.82385076, ..., -0.41705317,
         -0.44417109, -0.38577852],
        [-1.03922566, -1.12607456, -0.47089069, ..., -0.41705317,
         -0.44417109,  2.59216092],
        ...,
        [-0.90013245, -1.28446303,  0.23227555, ..., -0.41705317,
         -0.44417109, -0.38577852],
        [ 0.3501129 ,  1.1351671 ,  0.83714974, ..., -0.41705317,
         -0.44417109, -0.38577852],
        [ 0.72135149, -1.13748039, -1.21186157, ..., -0.41705317,
          2.25138468, -0.38577852]]))
```

In [210…

```python
# 11. LOGISTIC REGRESSION
lr = LogisticRegression(max_iter=5000, solver='saga')
lr.fit(X_train_res, y_train_res)
y_pred_lr = lr.predict(X_test_scaled)
y_pred_lr
```

Out[210… `array([0, 0, 0, ..., 0, 0, 0], dtype=int64)`

In [212…

```python
# 12. RANDOM FOREST
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train_res, y_train_res)
y_pred_rf = rf.predict(X_test_scaled)
y_pred_rf
```

Out[212… `array([0, 0, 0, ..., 0, 0, 0], dtype=int64)`

In [213…

```python
# 13. EVALUATION FUNCTION
def evaluate(y_test, y_pred, model_name):
    print(f"\n===== {model_name} =====")
    print("Accuracy:", accuracy_score(y_test, y_pred))
    print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
    print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

In [214…

```python
# 14. RESULTS
evaluate(y_test, y_pred_lr, "Logistic Regression")
evaluate(y_test, y_pred_rf, "Random Forest")
```

```
===== Logistic Regression =====
Accuracy: 0.97355

Confusion Matrix:
 [[19467   333]
 [  196     4]]

Classification Report:
               precision    recall  f1-score   support

           0       0.99      0.98      0.99     19800
           1       0.01      0.02      0.01       200

    accuracy                           0.97     20000
   macro avg       0.50      0.50      0.50     20000
weighted avg       0.98      0.97      0.98     20000


===== Random Forest =====
Accuracy: 0.9877

Confusion Matrix:
 [[19753    47]
 [  199     1]]

Classification Report:
               precision    recall  f1-score   support

           0       0.99      1.00      0.99     19800
           1       0.02      0.01      0.01       200

    accuracy                           0.99     20000
   macro avg       0.51      0.50      0.50     20000
weighted avg       0.98      0.99      0.98     20000
```

In [215…

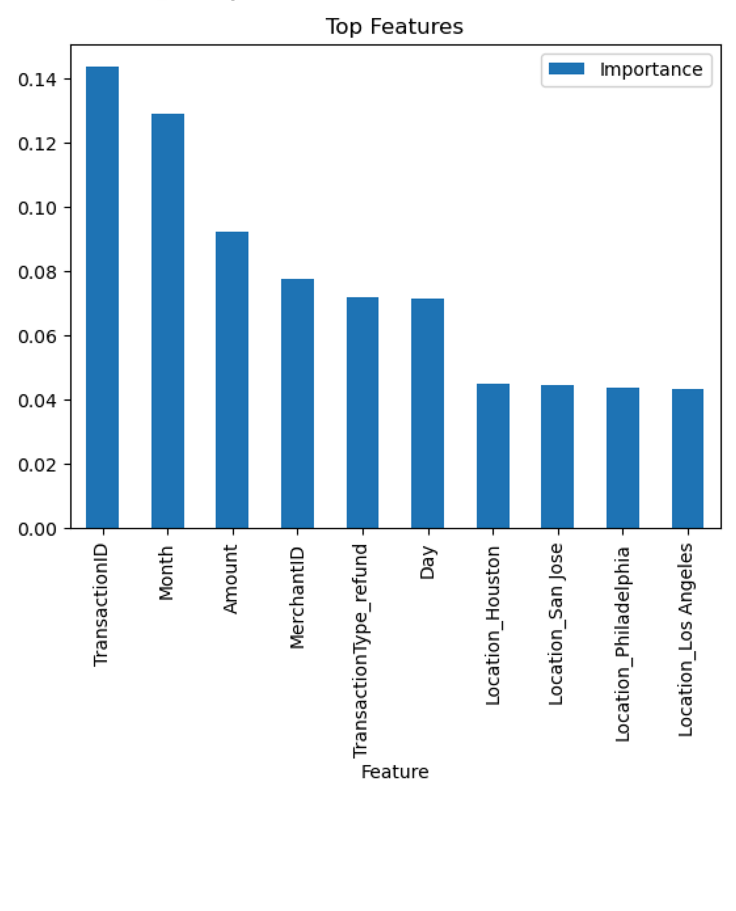
```python
# 15. FEATURE IMPORTANCE (RF)
importances = rf.feature_importances_

feat_df = pd.DataFrame({
    'Feature': X.columns,
    'Importance': importances
}).sort_values(by='Importance', ascending=False)

print("\nTop 10 Important Features:\n", feat_df.head(10))

feat_df.head(10).plot(kind='bar', x='Feature', title="Top Features")
plt.show()
```

```
Top 10 Important Features:
                   Feature  Importance
0           TransactionID    0.143629
4                   Month    0.128937
1                  Amount    0.092220
2              MerchantID    0.077533
6  TransactionType_refund    0.071860
5                     Day    0.071578
8        Location_Houston    0.044983
15      Location_San Jose    0.044566
11  Location_Philadelphia    0.043703
9    Location_Los Angeles    0.043458
```



** End Project **